

The Computing Stack (top → bottom)

Layer	What it is
You	Problems & ideas in natural language
Applications	Software: browsers, games, CMU CS Academy
Operating System	Windows, macOS, Linux. Manages resources & devices
Machine Language	Binary instructions the CPU runs
Hardware	CPU, RAM, storage, GPU
Logic Gates	AND, OR, NOT circuits
Transistors	Microscopic on/off switches

Abstraction: hiding the details of a lower layer so you can work with a simpler idea on top.

Hardware

CPU	Runs instructions. Its control unit directs the steps; its ALU does the math and comparisons.
RAM	Fast, temporary memory for running programs. Erased when power is off.
Storage	Long-term memory for files (SSD / hard drive).
GPU	Colors many pixels at once (works in parallel).

Fetch → Decode → Execute: the CPU’s cycle, repeated billions of times per second.

Key Terms

Binary — two digits: 1 = on, 0 = off.
Hardware — the physical parts. **Software** — programs + data that tell hardware what to do.
Logic gate — circuit whose output follows a rule for its inputs.
Computer science — the study of **computation**: how problems get solved step by step.

Language Ladder (human → machine)

#	Level	Example
1	Natural language	“Add abc and xyz...”
2	Pseudocode	total ← abc + xyz
3	High-level language	Python, Java, Scratch
4	Assembly language	LOAD ADD STORE
5	Machine language	10110000 01100001

High-level: easy to read, works on any hardware, less control.
Low-level: tied to one chip family, more control, harder to write.

Translating Code

Compiler	Translates the whole program before it runs (C, C++, Swift, Rust).
Interpreter	Translates & runs line by line as it goes (Python, JavaScript).
Assembler	Turns assembly into machine language.

Trade-offs

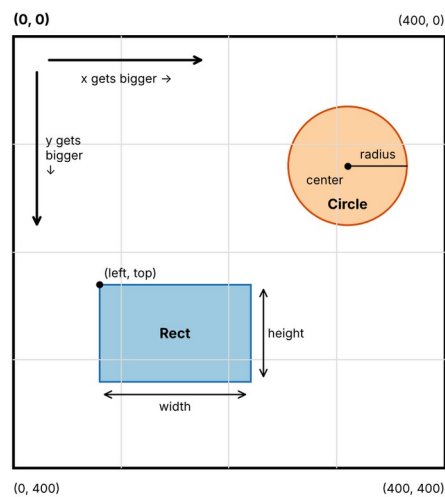
Close to hardware & precise ↔ abstract & less code
Compiled (fast) ↔ interpreted (flexible)
General-purpose ↔ specialized (SQL, R)
Text ↔ visual blocks (Scratch)

Which Language Fits the Job?

Language	Best for	Language	Best for
Python	Data, AI, quick scripts, bots	SQL	Asking questions of databases
JavaScript	Anything in a web browser	R	Statistics & research graphs
Java	Big teams, servers, lasts years	Rust	Fast + blocks memory bugs
C	Tiny chips, OS, life-critical devices	Go	Cloud servers, many tasks at once
C++	Game engines, max speed	Scratch	Beginners; no syntax errors
C#	Unity games, Windows apps	Assembly	First code a chip runs; total control
Swift	iPhone / Mac apps	PHP	WordPress & server web pages
Kotlin	Android apps	Lua	Roblox scripts, game mods
HTML & CSS	Page structure & looks (no logic)	COBOL	Old bank & government systems

Ask: Where does it need to run? How fast? Can it ever fail? Who is writing it?

The Canvas (400 × 400)



Finding Points

Edges: center ± radius (Circle) or center ± width/2, height/2 (Oval).

Rect center: (left + width/2, top + height/2)

Example: Oval(200, 300, 160, 80) → left 120, right 280, top 260, bottom 340.

Shapes

Shape	Arguments (in order)
Rect	left, top, width, height
Oval	centerX, centerY, width, height
Circle	centerX, centerY, radius
Line	x1, y1, x2, y2
Polygon	x1, y1, x2, y2, x3, y3, ... (3+ points)
RegularPolygon	centerX, centerY, radius, points
Star	centerX, centerY, radius, points
Label	value, centerX, centerY

Rules to Remember

- Code runs **top to bottom**. Later shapes are drawn **on top**.
- Commas between **every** argument.
- Names are **case-sensitive**: lineWidth not linewidth, None not none.
- Colors go in quotes: 'red'. None does not.
- Shapes can go off the canvas without an error. You just can't see them.
- fill=None with no border = invisible.

Shape Properties (optional, use name=value after the numbers)

Property	Example	Works on
fill	fill='red' · fill=None	All shapes
border	border='black'	All shapes
borderWidth	borderWidth=5	All shapes
gradient	fill=gradient('red', 'blue', start='top')	All shapes
lineWidth	lineWidth=10	Line
roundness	roundness=30	Star
size, font	size=30 · font='monospace'	Label
bold, italic	bold=True (thicker) · italic=True (slanted)	Label

Errors & Tools

Syntax error	Code breaks the rules of the language, so it won't run (missing comma, missing argument, wrong capital letter).
Logic error	Code runs, but does the wrong thing (shape in the wrong place, wrong color, wrong order).
Console	Shows error messages and the line number of the problem.
Inspector	Hover/click on the canvas to see coordinates and details about a shape.